Analysis and design of a platform for real-time speech translation

Jon Friis Jakobsen 123456789

Masters Project in Software Engineering

June 1887

The Maersk Mc-Kinney Moeller Institute
University of Southern Denmark

Abstract

Many companies are struggling to combine Java and Machine Learning...

Contents

Contents **ii**

1 Introduction **2**
1.1 Background and Motivation 2
1.2 Problem Statement 2
1.3 Research Questions 3
1.4 Success Criteria 4
1.5 Research Contribution and Scope 5
1.6 Thesis Structure and Question Traceability 5

2 Background **7**
2.1 State of the Art in Speech Translation 7
2.2 Microservices and ML Maintainability 7
2.3 Event-Driven Architecture (EDA) 8
2.4 AI Pipeline Components 9
2.5 Challenges in Distributed Architectures 9
2.6 Quality Assurance: From Testing to Evaluation 10
2.7 Synthesis and Design Implications 10

3 Analysis and Design **12**
3.1 Architectural Drivers 12
3.2 Quality Attribute Analysis 13
3.3 High-Level System Architecture 14
3.4 Data Modeling and Governance 15
3.5 Service Decomposition 16
3.6 Infrastructure and Deployment Strategy 17

4 Implementation **18**
4.1 Orchestrator and Event Flow 18
4.2 Data Management and the Claim Check Pattern 19
4.3 Model Wrapping and "Warm-Loading" 20
4.4 Orchestration and Reproducibility 21
4.5 Architectural Guardrails and VAD Integration 22

5 Evaluation **23**
5.1 Real-Time Responsiveness (Latency) 23
5.2 Heterogeneous Scalability Testing 23
5.3 Robustness and Chaos Engineering 23

5.4 Architectural Fidelity (Pipeline Quality) 23
5.5 Modifiability Proof 23

6 Conclusion **24**
6.1 Future Work 24

A Requirement Specifications **25**
A.1 Functional Requirements 25
A.2 Non-Functional Requirements 26

B Sequence Diagrams **28**

C Data Contracts (Avro Schemas) **29**
C.1 Base Envelope 29
C.2 Audio Ingestion 29
C.3 Speech Segmentation (VAD) 30
C.4 Translation Output 31
C.5 Synthesis Output (TTS) 32

Bibliography **33**

0

1 Introduction

1.1 Background and Motivation

Real-time speech-to-speech translation is motivated by a straightforward but technically demanding goal: reduce language barriers without forcing users into delayed, text-only, or manually mediated interaction. The societal relevance of multilingual access is well established in educational and public-service contexts, where language inclusion affects participation and access to knowledge [28]. At the same time, market reports and recent industrial overviews indicate strong commercial interest in speech translation, especially for cross-border collaboration, customer support, and meeting assistance [21, 16]. The combination of social demand and commercial momentum makes real-time translation an important applied research problem rather than a narrow laboratory exercise.

Current production-grade offerings are primarily provided by large cloud vendors through tightly integrated service portfolios such as Google Cloud Translation, Microsoft Translator, and the AWS speech stack built around Transcribe, Translate, and Polly [9, 20, 2, 3, 1]. These platforms demonstrate that speech translation has practical value, but they also embody architectural and operational assumptions that are difficult to inspect, adapt, or optimize for smaller actors, research prototypes, and open-source deployment scenarios. This creates a gap between what can be purchased as a managed capability and what can be studied as a transparent, reproducible, and extensible distributed system.

This thesis addresses that gap by investigating an event-driven microservice architecture for real-time speech-to-speech translation. Instead of treating translation as a single opaque model endpoint, the project frames the system as a pipeline of cooperating services for ingress, speech recognition, translation, speech synthesis, orchestration, and evidence collection. That framing is relevant because architectural choices about decoupling, message flow, observability, and artifact handling shape whether the system can achieve low latency, scale under load, and remain robust when components fail [22, 26, 6].

1.2 Problem Statement

The literature on speech translation is rich in model innovation, but comparatively thinner on deployable system architecture. Research has produced strong advances in neural machine translation, speech recognition, and direct

speech-to-speech translation, including transformer-based sequence modeling, large-scale weakly supervised ASR, speech-to-text toolchains, and recent direct translation pipelines [5, 29, 25, 30, 11, 27, 32]. However, these advances do not by themselves explain how multiple AI components should be composed into a distributed platform that must satisfy tail-latency constraints, preserve evidence integrity, and remain maintainable over time.

This gap becomes more pronounced when the system must be evaluated as an architecture rather than a benchmarked model. A real-time speech translation platform must coordinate data flow across ASR, MT, and TTS stages; isolate failures; make queueing and service boundaries observable; and support reproducible measurement of latency and robustness under realistic load [13, 10, 7, 31]. Existing work rarely treats these system-level concerns as the primary object of study. The central problem addressed in this thesis is therefore not only how to translate speech, but how to design and justify an event-driven architecture that makes such translation low-latency, scalable, robust, and empirically assessable.

1.3 Research Questions

The thesis is organized around one main research question and five supporting sub-questions.

Main Research Question

How can an event-driven microservice architecture be designed and implemented to deliver robust and scalable real-time speech-to-speech translation with measurable performance optimization?

Sub-questions

1. What architectural principles and design patterns support an efficient real-time speech translation pipeline?
2. What factors affect the system's latency and ability to scale?
3. How can these factors be measured and optimized in practice?
4. How can the system be made robust and available even if individual components fail?
5. How can the system's performance and quality be evaluated and possibly compared to existing solutions?

These questions are intentionally distributed across the thesis rather than answered inside the introduction alone. The current chapter frames the prob-

lem, scope, and evaluation logic. The background chapter establishes the relevant literature and design vocabulary. Later chapters address architectural response, implementation realization, and empirical evaluation in sequence.

1.4 Success Criteria

The thesis evaluates success across architectural, empirical, and methodological dimensions.

Real-Time Success

For this project, *real-time success* is defined by the system's ability to process speech input and return translated output with an end-to-end latency target of P99 below two seconds under the scoped MVP conditions. This criterion does not claim universal real-time performance across all workloads; instead, it provides a concrete architectural driver that shapes service decomposition, observability requirements, and the later evaluation strategy. In practical terms, low median latency is insufficient if tail latency remains unstable, because the user experience of conversation is dominated by worst-case delays rather than best-case responses.

Scientific Rigor and Evidence Handling

The thesis also treats *scientific rigor* as a success criterion. Performance and robustness claims must be grounded in reproducible evidence generated through the project's uv-first validation workflow, so that the later implementation and evaluation chapters can refer to repeatable procedures rather than ad hoc one-off measurements. At the same time, manuscript-facing evidence must remain privacy-safe. This means that shareable evidence is restricted to metadata and aggregated observations, such as percentile latency summaries, correlation identifiers, event types, and bounded artifact sizes, while raw transcripts, raw translated text, raw audio, credentials, and full bearer-style URLs are excluded from the manuscript narrative and supporting handoff materials. In particular, latency discussion in this thesis relies on aggregated metrics such as P50, P95, and P99 rather than raw per-request timestamps.

Evaluation Orientation

Success is finally assessed through the combination of latency, scalability, robustness, and quality evaluation. The introduction therefore frames the thesis as a systems investigation in which translation quality matters, but is interpreted together with architectural consequences such as service-boundary overhead, warm-versus-cold execution behavior, and the observability needed to explain measured outcomes. The later evaluation chapter is responsible for

validating these criteria empirically; the introduction only defines the standard against which the system will be judged.

1.5 Research Contribution and Scope

Research Contribution

The contribution of the thesis is a defensible reference architecture for real-time speech-to-speech translation that connects distributed-systems design choices to measurable performance trade-offs. More specifically, the work contributes: (1) an architectural framing for event-driven composition of ASR, MT, and TTS services; (2) a maintainability-focused discussion of service boundaries, observability, and contract evolution in AI pipelines; and (3) an evaluation-oriented treatment of how latency, scalability, and robustness claims can be supported with reproducible and privacy-safe evidence [6, 7, 31]. The contribution is therefore not a new translation model, but a structured architectural and empirical argument for how an open, inspectable platform can be designed and assessed.

Scope Boundaries

The project includes the design and implementation of an event-driven platform built from existing AI components, together with performance, scalability, and robustness evaluation under the scoped MVP conditions. It does not include training new foundation models, exhaustive legal or organizational analysis, full production hardening, or a general-purpose comparison against every commercial alternative. Security is treated here as an evidence-governance and privacy-by-design constraint for manuscript-facing artifacts, while deeper runtime security engineering remains the concern of later implementation and deployment work.

1.6 Thesis Structure and Question Traceability

The remainder of the thesis is organized to answer the research questions progressively. Chapter 2 establishes the literature context for speech translation and event-driven systems. It also introduces AI pipeline maintainability and distributed-systems trade-offs. The Analysis and Design chapter translates that context into architectural drivers, quality attributes, service decomposition, and data-governance choices. The Implementation chapter explains how the platform was realized in practice, including orchestration, artifact handling, and robustness mechanisms. The Evaluation chapter measures the system against the success criteria defined above. The Conclusion then synthesizes the final answer to the main research question.

This structure also makes the sub-question ownership explicit. Sub-question 1 is motivated in the Background chapter and answered architecturally in Analysis and Design. Sub-questions 2 and 3 are framed here first. They are then operationalized through design and implementation and resolved empirically in Evaluation. Sub-question 4 is addressed through implementation and robustness evidence. Sub-question 5 depends on the evaluation and conclusion chapters. The introduction therefore functions as a scope-locking and traceability chapter, ensuring that later arguments remain tied to the original research objective.

2 Background

2.1 State of the Art in Speech Translation

Speech translation research spans both modular pipelines and direct end-to-end approaches. In the modular tradition, speech is transcribed, translated, and optionally synthesized in separate stages, making it possible to combine specialized ASR, MT, and TTS components that can each be improved independently [5, 29, 25, 30]. This decomposition remains attractive in practice because it exposes explicit control points for error handling, observability, fallback behavior, and selective optimization.

More recent work on direct speech-to-speech translation demonstrates that tightly integrated models can reduce intermediate representation overhead and potentially improve naturalness in some settings [11, 27, 32]. Even so, direct approaches do not eliminate the architectural questions relevant to this thesis. A production-oriented platform still needs to orchestrate ingress, buffering, session context, failure handling, artifact persistence, and evaluation evidence. In other words, model architecture and system architecture solve different layers of the same problem.

Industrial offerings confirm the practical importance of the problem space but do not by themselves resolve the research gap. Cloud platforms from Google, Microsoft, and Amazon provide mature translation services, yet they package architectural decisions behind proprietary boundaries [9, 20, 2, 3, 1]. Recent applied studies show that real-time translation is increasingly feasible, but they also underscore the continued need to reason about the whole pipeline rather than isolated models [23, 17]. This thesis therefore positions itself between component-level research and black-box managed services by focusing on the architecture needed to make a transparent, distributed speech translation system workable.

2.2 Microservices and ML Maintainability

Microservices are relevant to this thesis because they offer a way to separate concerns that scale differently and fail differently. Newman describes microservices as independently deployable services organized around business or domain capabilities, while Richardson emphasizes explicit boundaries, contracts, and asynchronous interaction patterns as mechanisms for preserving autonomy over time [22, 26]. For a speech translation platform, this matters because audio ingress, speech recognition, machine translation, speech synthe-

sis, orchestration, and persistence rarely share the same performance profile or operational lifecycle.

Maintainability becomes even more important when AI components are involved. Engineering AI systems requires attention not only to code modularity, but also to data movement, model packaging, runtime dependencies, reproducibility, and observability across model-serving boundaries [7]. Xu et al. further highlight that scalability and maintainability challenges in machine learning systems often arise from the interaction between model pipelines and their surrounding operational infrastructure, not from the model itself [31]. A microservice architecture does not automatically solve these problems, but it provides a structure in which concerns such as independent scaling, versioned interfaces, and controlled failure domains can be made explicit and studied.

The trade-off is that decomposition introduces coordination overhead. A microservice-based AI pipeline gains modularity and clearer ownership, but it also introduces network hops, serialization costs, and observability requirements that would be less visible in a monolith. This tension between modular maintainability and latency overhead is one of the central architectural trade-offs examined in the thesis.

2.3 Event-Driven Architecture (EDA)

Event-driven architecture is the coordinating paradigm adopted in this thesis because it decouples producers and consumers while allowing the pipeline to remain observable and extensible. In data-intensive systems, logs and streams provide a durable coordination mechanism that can support replay, asynchronous processing, and bounded service autonomy [13]. Enterprise integration patterns further describe how messages, envelopes, and broker-mediated routing can be used to connect heterogeneous components without forcing direct synchronous dependencies [10].

For speech translation, an event-driven design is attractive because the output of one stage naturally becomes the input of the next. Speech recognition emits text events, translation consumes and enriches those events, and synthesis produces output artifacts for return to the client. This pattern supports service isolation and flexible scaling, while also making it possible to trace processing across stage boundaries through correlation identifiers and hop-to-hop duration measurements. Brokered event streams and contract-driven event exchange are therefore not incidental implementation details; they are part of the architectural rationale for how a distributed AI pipeline can remain diagnosable under load [13, 26].

The same literature also motivates two specific patterns that matter later in the thesis. First, the *claim-check* pattern allows large artifacts such as audio files to be stored out of band while lightweight references move through the event stream [10]. In a secure system, those references must be bounded and

short-lived rather than permanent bearer credentials. Second, schema governance is needed so that event producers and consumers can evolve without breaking the pipeline. In the context of this thesis, schema evolution is treated as a high-level contract-governance concern: event changes must preserve interoperability and evidence traceability, not merely serialization success.

2.4 AI Pipeline Components

The platform examined in this thesis composes several AI-facing components into one end-to-end workflow. Automatic speech recognition converts audio into text, machine translation transfers meaning across languages, and text-to-speech converts translated text into audible output. Voice activity detection and ingress/orchestration logic support these stages by segmenting input, reducing unnecessary compute, and coordinating the conditions under which downstream services should act.

Each component family has its own research and engineering concerns. Whisper-style ASR emphasizes robustness across noisy and multilingual inputs [25]. Transformer-based MT and related sequence models provide the translation backbone used in many modern systems [5, 29]. Speech-to-text and direct speech-translation work show how model architectures can be tuned for latency and multilingual operation, but they do not remove the need for surrounding system decisions about batching, warm starts, queue discipline, and failure containment [30, 11, 32]. The architecture must therefore be studied as the environment in which these AI components become a usable platform.

This component view also supports the thesis' maintainability argument. Treating ASR, MT, TTS, VAD, and orchestration as bounded but cooperating services makes it easier to reason about resource isolation, deployment strategy, and selective optimization. It also creates explicit integration points where observability and evidence collection can be attached without requiring the system to be redesigned as a single model-serving endpoint.

2.5 Challenges in Distributed Architectures

Distributed speech translation is constrained by more than raw model accuracy. Tail latency accumulates across network calls, queueing delays, serialization boundaries, and service startup behavior. A system that appears fast in isolated component benchmarks can still fail to feel real-time when exposed to concurrency, warm-up effects, or failure recovery paths. This is why the thesis treats P99 latency, rather than only average response time, as an architectural driver.

Scalability introduces a second challenge. Different pipeline stages scale according to different resource bottlenecks: ASR and TTS may be GPU-bound, orchestration may be coordination-bound, and persistence or broker infrastruc-

ture may become throughput-sensitive under burst load. Distributed-systems literature shows that scaling decisions are inseparable from partitioning strategy, observability, and failure semantics [13, 6, 15]. For this reason, the thesis evaluates scalability as a system property of the whole pipeline rather than as a proxy for the throughput of any single component.

Robustness and diagnosability form the third challenge. A multi-stage AI pipeline must continue to behave predictably when one stage slows down, restarts, or emits malformed data. Correlation identifiers, bounded message contracts, and hop-to-hop timing are essential because they allow failures to be localized and explained. At the same time, observability must remain privacy-safe. In this thesis, performance analytics are treated as metadata-only evidence: aggregated percentiles, event types, and duration summaries are admissible in the manuscript, while raw user content and raw request traces are not. That constraint is part of the research design rather than an afterthought.

2.6 Quality Assurance: From Testing to Evaluation

The background for this thesis must distinguish between testing a system and evaluating a research claim. Testing establishes whether the platform behaves as intended under controlled scenarios. Evaluation interprets the resulting evidence against the research questions and success criteria. In a distributed AI pipeline, both are necessary: without testing there is no trustworthy basis for evidence, and without evaluation there is no defensible thesis argument.

Reproducibility is therefore central. The project adopts a uv-first validation workflow so that later claims about latency, robustness, and integration behavior can be tied to repeatable execution rather than undocumented local setups. That workflow matters academically because it narrows the gap between implementation detail and thesis evidence. At the same time, only privacy-safe evidence belongs in the manuscript. This is why the evidence posture distinguishes between shareable metadata and non-shareable debug information, and why later empirical claims must be supported with aggregated measurements and sanitized artifacts rather than raw logs or secret-bearing command examples.

This distinction also clarifies the role of the later evaluation chapter. The background chapter can justify why reproducibility, observability, and privacy-safe evidence handling are necessary. It does not yet claim that the platform satisfies the target metrics; that remains an empirical question for later chapters.

2.7 Synthesis and Design Implications

The literature reviewed in this chapter supports a clear design direction. Speech translation is no longer a speculative application area. The research gap has

shifted from individual model capability toward architectural composition and empirical system behavior. Microservices provide maintainability and bounded ownership. Event-driven coordination supports loose coupling and diagnosability. Distributed-systems theory explains why latency, scalability, and robustness must be treated together rather than independently.

These observations motivate the design choices developed in the next chapter. The thesis therefore carries forward six implications: (1) the pipeline should be decomposed into explicit service stages; (2) event flow should be traceable across stage boundaries; (3) large artifacts should move through claim-check style references rather than bulk messages; (4) contract evolution must preserve interoperability and evidence continuity; (5) evaluation evidence must be reproducible; and (6) manuscript-facing evidence must remain privacy-safe and aggregated. Together, these implications connect the literature review to the architectural response and make the later chapter sequence answerable against the research questions introduced in Chapter 1.

3 Analysis and Design

3.1 Architectural Drivers

This chapter converts the design implications from Chapter 2 into an explicit architectural response for the speech translation platform. The objective is not merely to list components, but to explain why the chosen structure is a defensible answer to the requirements stated in the requirements specification. The primary drivers are therefore derived from the platform's measurable constraints and evidence obligations rather than from technology preference alone. In particular, the architecture must satisfy real-time interaction goals, preserve diagnosability across a distributed pipeline, and remain maintainable enough to evolve as models and deployment targets change [13, 26, 6].

Latency and interaction quality. The first driver is end-to-end responsiveness. The platform must deliver first useful output within three seconds and keep segment-level latency within the targets captured by `NFR-001` and `NFR-002`. That requirement rules out an architecture that waits for complete utterances before producing value. Instead, the design combines early voice activity detection (`FR-024`), partial ASR output (`FR-007`), streaming translation (`FR-008`), and continuous TTS output (`FR-009`). Compute-heavy stages are explicitly allowed to run as Ray Actors (`FR-021`) so that models can stay warm in memory instead of paying repeated cold-start costs.

Scalability and change isolation. The second driver is independent scaling and bounded evolution. Audio ingress, orchestration, ASR, MT, and TTS do not have the same runtime profile, and the requirements explicitly call for asynchronous service-to-service communication (`FR-012`) and per-service horizontal scaling (`FR-020`). This pushes the design toward event-driven microservices in which each stage can scale or be replaced without forcing the entire pipeline to be redeployed. Schema governance (`FR-013`) and contract stability are therefore not secondary implementation concerns; they are architectural enablers that allow the platform to evolve while preserving interoperability.

Reliability, auditability, and privacy-safe evidence. The third driver is trustworthy operation under failure and under evaluation. Retry and controlled degradation (`FR-015`), trace propagation (`FR-014`), artifact storage (`FR-010`), retention control (`FR-011`), and privacy-safe observability (`NFR-004` and `NFR-005`) together require an architecture that can explain what happened without exposing sensitive content. This is why the platform uses correlation and trace metadata as first-class design elements, treats transcripts and audio as sensitive artifacts, and separates shareable normal telemetry from

opt-in debug telemetry. The chapter is thus driven by the intersection of performance, operational control, and evidence discipline rather than by throughput alone [7, 31].

3.2 Quality Attribute Analysis

The quality attributes for the platform are tightly coupled. Improving one dimension, such as modularity, can easily worsen another, such as latency. The design therefore prioritizes a small set of explicit tactics that can be defended against the requirements and against the distributed-systems literature.

Performance. Performance is shaped by the fact that speech translation is a pipeline rather than a single call. The architecture reduces avoidable latency in three ways. First, voice activity detection is used as a gatekeeper so silence and noise do not consume expensive ASR capacity. Second, partial results are propagated through ASR and MT so the UI can display and speak useful output before the user has finished a full utterance. Third, the compute-heavy stages are isolated so warm-loaded Ray Actors can be used where startup overhead is otherwise dominant. The trade-off is that the system accepts more coordination complexity in exchange for lower interaction delay.

Scalability and modifiability. Scalability is not treated as a single global knob. Instead, the design assumes that the gateway, orchestrator, broker, storage, and inference services will scale differently. Event-driven integration allows these stages to be replicated or tuned independently, and the use of schema-managed events keeps those boundaries explicit. This increases operational overhead compared with a monolith or a direct synchronous chain, because messages must be serialized, validated, and traced. The benefit is that the architecture can absorb changes in one stage, for example a different ASR runtime or a new translation model, without rewriting the rest of the pipeline [13, 22, 26].

Robustness, auditability, and privacy. Robustness in this platform is inseparable from diagnosability. A service restart, malformed event, or storage outage must be observable through `correlation_id`, `traceparent`, event type, and typed error metadata rather than through raw payload logging. This requirement leads directly to a telemetry policy in which normal mode is always on, low volume, and safe to share, while debug mode is explicit, higher volume, and still forbidden from logging raw audio, transcripts, translations, or credentials. The architecture therefore uses metadata-rich but content-poor telemetry as its baseline evidence posture. That constraint is academically useful because it aligns runtime observability with the thesis requirement for reproducible and privacy-safe evidence.

Security as an architectural quality attribute. Security is addressed as a design constraint rather than a post-processing layer. Session-bound token issuance and the WebSocket AUTH control message protect the ingress

path, while internal broker traffic is modeled as an internal-only channel with ACL and mTLS enforcement. For persistent data, MinIO is used with claim-check references and server-side encryption expectations so that large artifacts can be retained for auditability without turning every internal event into a bearer-secret container. The cost is additional governance and configuration complexity, but the resulting design is more defensible because it states where trust boundaries actually exist.

3.3 High-Level System Architecture

At a high level, the platform is organized around a thin client-facing ingress layer, an asynchronous event backbone, a small set of bounded AI pipeline services, and persistent stores for state and artifacts. The browser interacts with the system through two ingress surfaces. A REST API is used for session creation, join flow, and token issuance, while a WebSocket endpoint is used for low-latency bidirectional exchange of audio and control messages. This separation keeps short-lived control and long-lived streaming concerns distinct, while still allowing both to participate in the same session context.

Once a session is established, the WebSocket server does not directly call downstream AI services in a synchronous chain. Instead, it emits audio input events onto the broker, where the pipeline is advanced by event consumers and coordinated by a lightweight orchestrator. The broker is the architectural abstraction; Kafka is the chosen implementation. This distinction matters because the chapter's argument concerns decoupled event flow and bounded consumers, not loyalty to one specific product. The orchestrator exists to preserve ordering, correlation, and state progression across a fundamentally asynchronous system, thereby satisfying `FR-012` without collapsing the design back into direct REST coupling [13, 10].

The main processing path follows a clear sequence. The gateway publishes an *AudioInputEvent*. The VAD service filters silence and emits a *SpeechSegmentEvent*. ASR consumes those segments and emits recognized text. MT consumes text and emits translated text. TTS consumes the translated text and emits synthesized audio metadata. The WebSocket server then delivers transcript updates and binary TTS output back to the client. Because these boundaries are explicit, the platform can attribute latency and failure to individual hops instead of treating the whole system as a single opaque inference call.

Large payloads are deliberately kept off the event stream where possible. Audio recordings, segments, transcripts, and synthesized output can be persisted to object storage and referenced through claim-check style handles. This keeps the broker focused on coordination and state progression, while persistent stores carry the heavier audit and retention burden. The result is an architecture in which synchronous control is limited to the user-facing edge,

and internal pipeline progress is coordinated asynchronously.

3.4 Data Modeling and Governance

The platform's core data model consists of four logical units: the session, the segment, the event, and the artifact. A session captures the conversational context, selected languages, and security context. Segments represent bounded processing units derived from the live stream. Events carry lightweight state transitions between services. Artifacts represent the larger persistent outputs, such as raw audio, segmented audio, transcripts, translations, and TTS waveforms. This separation is important because the system has different governance obligations for coordination metadata than for user-derived content.

Traceability model. The architectural spine is `correlation_id` plus `traceparent`. The session identifier provides user-facing continuity, but correlation metadata provides the operational continuity that allows one request path to be followed across gateway, broker, orchestrator, inference services, and storage. This satisfies `FR-014` and also supports the evidence posture described elsewhere in the project documentation: shareable artifacts should be reconstructible from metadata and deterministic identifiers rather than from raw content dumps.

Claim-check storage model. For auditability, the system stores central pipeline outputs in MinIO with deterministic key structures organized around correlation and segment identity. Internal events carry non-presigned object references rather than bearer-style URLs, which satisfies `NFR-007`. Presigning, when needed at all, is treated as an edge concern rather than an internal integration pattern. This is a direct application of the claim-check pattern: the broker carries references and coordination metadata, while the object store carries large and potentially sensitive payloads [10]. The documented retention baseline is 30 days, which aligns the storage model with `FR-011` and with the broader evidence-lifecycle governance used in the project.

Privacy-safe telemetry baseline. Governance also applies to observability data. The normal telemetry mode is intentionally allowlisted to low-volume, stable fields such as `correlation_id`, component identity, event type, language metadata, payload sizes, and duration measurements. It explicitly forbids raw audio bytes, transcript text, translated text, query strings, presigned URLs, and credentials. Debug telemetry remains opt-in and non-shareable, but even there the design still avoids raw content and secrets. This distinction is necessary because the same platform must support both runtime diagnosis and thesis evidence packaging without leaking personal data or bearer tokens.

Security boundaries. The governance model also clarifies trust boundaries. Persistent artifacts are expected to use MinIO server-side encryption. Broker traffic is treated as internal-only and protected through ACL and mTLS

assumptions. Local development defaults to loopback-bound published ports to avoid accidental LAN exposure. Together these choices make the data path easier to defend academically: the chapter can state where sensitive content is stored, how it is referenced, how long it is retained, and which telemetry classes are safe to share.

3.5 Service Decomposition

The platform is decomposed into services according to operational role and scaling behavior rather than according to the internal boundaries of any one model. This decomposition is intended to make the system understandable, replaceable, and measurable.

- **API gateway.** The gateway exposes REST and WebSocket interfaces, terminates the client protocol boundary, validates the initial authentication flow, and translates browser interaction into broker-visible events. It is intentionally thin so user-facing protocol concerns do not leak into inference services.
- **Pipeline orchestrator.** The orchestrator maintains ordering and state progression for the end-to-end workflow. Its role is not to perform inference, but to ensure that asynchronous stages still behave like one coherent pipeline for a given correlation context.
- **VAD service.** VAD is isolated because it acts as a protective efficiency boundary. By filtering silence before ASR, it preserves compute for meaningful speech and reduces noise-induced downstream churn.
- **ASR, MT, and TTS services.** These are the core AI stages. They are separated because they have different runtime dependencies, different scaling profiles, and different opportunities for model substitution. ASR and TTS are especially strong candidates for Ray Actor execution due to warm-model reuse.
- **Broker and schema layer.** The broker provides temporal decoupling and routing, while schema governance protects compatibility between producers and consumers. Together they form the integration contract of the system.
- **Persistent stores.** PostgreSQL stores session-oriented application state, while MinIO stores larger pipeline artifacts. Separating relational state from object artifacts avoids forcing one storage engine to perform both roles poorly.

This decomposition is deliberately more granular than a simple API-plus-model monolith, but it is not decomposition for its own sake. Each boundary

corresponds to a distinct reason to scale, secure, observe, or evolve that stage differently. The orchestrator and gateway remain comparatively light-weight, whereas the inference services are isolated so they can exploit GPU resources or Ray runtime features without dragging those dependencies into every other component. The resulting design is therefore aligned with maintainability concerns from the literature and with the requirements for independent scaling and diagnosability [22, 7].

3.6 Infrastructure and Deployment Strategy

The deployment strategy follows the same principle as the service decomposition: choose the simplest operational arrangement that still makes the architectural claims testable. Docker Compose is the baseline environment for reproducible local development, smoke testing, and evidence generation. It supports `FR-018` by making the full stack startable for a new contributor, and it keeps the architecture observable in one place during early validation.

For distributed deployment, Docker Swarm is the release-target topology. This satisfies `FR-019` and `FR-020` without introducing the additional operational overhead that a Kubernetes-first strategy would impose on a research project. The choice is therefore pragmatic rather than ideological: Swarm is sufficient to demonstrate replicated services, internal overlay networking, secret distribution, and environment-specific configuration while keeping the deployment argument understandable within the thesis scope. Infrastructure-as-code through Pulumi remains a post-MVP extension path rather than a prerequisite for the architectural core.

Security and evidence handling are also reflected in the deployment model. Internal services such as Kafka, Schema Registry, and MinIO are treated as internal-only surfaces. Local development binds user-facing ports to loopback, and cluster-oriented deployment expects explicit secret handling for storage and broker credentials. This is consistent with the chapter's broader claim that deployment strategy is not separate from architecture: network boundaries, storage trust assumptions, and telemetry policies all shape what the system can safely prove about itself.

Taken together, the Compose-to-Swarm progression supports the value statement of the epic. It shows how the platform can be reasoned about at both development and deployment time, how the microservice boundaries map onto real operational units, and why the event-driven design remains the most coherent response to the combined latency, scalability, robustness, and auditability requirements.

4 Implementation

The implementation of the speech translation platform translates the event-driven microservices architecture defined in Chapter 3 into a functional, distributed system. This chapter narrates the realization of the orchestrator logic, the management of heavy audio artifacts via the Claim-Check pattern, the deployment of model actors using Ray, and the operational hardening that ensures reproducibility and security.

4.1 Orchestrator and Event Flow

The platform's control plane is governed by a Python-based orchestrator utilizing the FastAPI framework to manage asynchronous task lifecycles. The orchestrator serves as the primary consumer of the decision log, mapping high-level user requests to specific service-chain invocations.

Event-Driven Logic and Kafka Integration

The orchestrator implements the architectural decision to decouple inference services via an event broker. Using the `confluent-kafka` library, the orchestrator monitors the `speech.orchestration.command` topic for incoming ASR results or translation requests. Upon receiving an event, the orchestrator consults the session state and dispatches the subsequent command (e.g., initiating machine translation after speech-to-text completion). This reactive model allows the platform to handle bursts of concurrent speaker streams without maintaining persistent, blocking socket connections throughout the entire AI pipeline.

Least-Privilege Enforcement via Topic ACLs

To maintain architectural integrity and prevent data leakages between service boundaries, the Kafka implementation follows a strict least-privilege model. Each service identity is restricted to specific topics through Access Control Lists (ACLs). For instance, the `asr-service` is permitted only to publish to the command topic and consume from its dedicated input stream, while the `orchestrator-service` is the sole identity authorized to consume from command topics and publish to decision topics. This implementation ensures that a compromise of a single model actor does not grant broad access to the entire orchestration flow (FR-011).

Observability: Normal-vs-Debug Telemetry

Adhering to the project's privacy-safe observability policy, the orchestrator implements a dual-mode telemetry strategy. In *Normal Mode*, the orchestrator emits only metadata-intensive logs and distributed traces. These include `correlation_id` propagation (linking ASR, MT, and TTS spans), processing durations, and byte counts. Crucially, raw audio content or transcriptions are excluded by default to allow for shareable evidence logs in an academic context. *Debug Mode* can be opted-in via environment variables, enabling deeper payload inspection for local development, provided it emits a clear security warning at startup. This separation ensures that the "Den Gyldne Rengøringsregel" is enforced: the system remains diagnosable in production without violating user privacy.

4.2 Data Management and the Claim Check Pattern

The platform handles significant volumes of raw audio data, which presents a challenge for traditional message broker payloads. To address this, the implementation utilizes the Claim-Check pattern, decoupling large binary artifacts from the event orchestration stream.

Artifact Persistence via MinIO

The primary storage layer is implemented using MinIO, an S3-compatible object storage server. When a client initiates a speech stream at the Ingress Gateway, raw audio chunks are captured and persisted directly to a dedicated bucket. Instead of transmitting the heavy audio bytes through the Kafka orchestration topics, the platform generates a unique object reference (the "claim"). This reference is then passed to subsequent services (ASR, MT, TTS) in the event payload, allowing them to fetch only the necessary segments on demand. This approach significantly reduces Kafka overhead and ensures that the event log remains lean and focused on control flow rather than data transport (NFR-007).

Claim Lifecycle and Pre-signed URL Hardening

To ensure secure and efficient access to audio artifacts, the platform implementation leverages S3 pre-signed URLs. When a service like `asr-service` needs to process an audio segment, the orchestrator provides a temporary URL with minimal access permissions. To mitigate the risk of bearer token leakage, the platform enforces a strict **Short-lived URL Policy**, with Time-To-Live (TTL) values restricted to less than one hour. This ensures that even if a

URL is intercepted, its window of validity is insufficient for meaningful misuse, thereby hardening the implementation against unauthorized data access.

Data Encryption at Rest (SSE-S3)

The MinIO implementation serves as the primary security boundary for audio data. All artifacts are encrypted at rest using **Server Side Encryption (SSE-S3)** with AES-256 encryption. This implementation ensures that the "Clean Boundary" between services is maintained at the storage layer; even in the event of unauthorized access to the underlying storage disks, the raw audio payloads remain unintelligible. The encryption keys are managed within the Swarm infrastructure, ensuring that the platform meets the highest standards of data protection for both normal and debug execution runs.

4.3 Model Wrapping and "Warm-Loading"

The serving of heavy AI models (Whisper and SeamlessM4T) is managed via the Ray framework, which provides a high-performance substrate for distributed model inference. Each model is wrapped within a Ray Actor, allowing the platform to maintain persistent session state and GPU allocation for individual requests.

Ray Actors and "Warm-Loading" Strategy

A critical implementation choice for meeting the platform's latency goals (P99 < 2s) is the use of a "Warm-Loading" strategy. Unlike traditional lambda-style serverless functions that incur significant cold-start penalties for model initialization, the Ray implementation pre-loads the model weights into GPU memory during service startup. This optimization (NFR-004) enables the platform to respond to translation requests with minimal head-of-line blocking, as the inference pipeline is "warm" and ready for immediate computation upon receiving a new Kafka event.

Dynamic Resource Allocation

The Ray framework facilitates dynamic resource management, allowing the platform to scale model actors based on GPU availability and request pressure. Each `asr-actor` is constrained to a specific GPU fractional allocation (e.g., 0.5 GPU), enabling multiple actors to share a single high-performance card while maintaining isolation (FR-014). This approach is essential for scaling the platform from a local MVP to a distributed Docker Swarm environment where GPU resources are shared across multiple nodes.

4.4 Orchestration and Reproducibility

The transition from a local development environment to a production-grade distributed cluster is a critical component of the platform's feasibility proof. This section narrates how the implementation ensures environment parity and reproducibility.

Transition from Docker Compose to Swarm

The platform development lifecycle begins in a local environment using `docker-compose`, allowing for rapid iteration and debugging. To scale for actual evidence runs, the implementation transitions to **Docker Swarm**. This transition is facilitated by a unified configuration baseline, where the same root `.env` file serves as the single source of truth for both environments. This **Environment Parity** ensures that architectural invariants, such as service-to-service communication paths and network configurations, remain consistent regardless of the underlying orchestration layer (FR-020).

Security Hardening in Swarm: Secrets and Overlays

The production implementation in Swarm introduces additional hardening measures. Sensitive credentials (e.g., Kafka SASL keys, MinIO S3 secrets) are managed via **Docker Swarm Secrets**, ensuring they are never stored in plain text or committed to the repository. Furthermore, the platform utilizes **Swarm Overlay Networks** to enforce network segmentation. Stateful backend services (MinIO, Kafka) are isolated on a dedicated `speech_backend` network, inaccessible from the public-facing `speech_gateway` through the Ingress Gateway (NFR-009). This layering ensures that the implementation matches the architectural security drivers while remaining shareable for thesis evaluation.

Requirement Traceability

The final proof of the platform's feasibility is recorded in the traceability matrix. Every component narrated in this chapter—from the orchestrator's event logic to the Ray model actors—maps directly to the functional and non-functional requirements defined in the specification. For instance, the Claim-Check pattern implementation via MinIO and Kafka directly realizes the secure artifact management requirement (FR-007), while the Warm-Loading Ray actors fulfill the latency-bound inference objective (NFR-004). This mapping ensures that the thesis manuscript provides a complete and reviewable evidence path for the evaluator.

4.5 Architectural Guardrails and VAD Integration

To ensure the robustness of the inference pipeline, the implementation incorporates specific architectural guardrails. Voice Activity Detection (VAD) is integrated as a pre-processing step to filter silence and non-speech artifacts before they reach the more compute-intensive ASR and translation stages. This VAD implementation serves as a crucial latency-saving measure, as it prevents the heavy inference models from being invoked for empty or noisy audio segments, thereby optimizing overall system throughput and preventing GPU bottlenecks (NFR-006).

5 Evaluation

5.1 Real-Time Responsiveness (Latency)

5.2 Heterogeneous Scalability Testing

5.3 Robustness and Chaos Engineering

5.4 Architectural Fidelity (Pipeline Quality)

5.5 Modifiability Proof

6 Conclusion

6.1 Future Work

A Requirement Specifications

This appendix contains the complete list of Functional (FR) and Non-Functional (NFR) requirements defined for the Universal Speech Translation Platform.

A.1 Functional Requirements

ID	Description	Priority
FR-001	**Session Management:** The system must support creating, starting, and stopping a session.	MUST
FR-002	**Language Support:** The system must support dynamic language selection per session. MVP minimum: da, en, de, fr, es to English.	MUST
FR-003	**Language Switching:** The system should allow changing language pairs during an active session.	SHOULD
FR-004	**Microphone Stream:** The system must support live audio ingestion from a microphone via a gateway.	MUST
FR-005	**Audio Upload:** The system must support uploading audio chunks as an alternative to streaming.	MUST
FR-006	**Segment & Streaming:** The system must process both discrete segments and continuous streams.	MUST
FR-007	**Partial ASR Results:** ASR must produce intermediate (partial) transcriptions before the final result.	MUST
FR-008	**Streaming Translation:** The translation component must update translations continuously as text evolves.	MUST
FR-009	**Streaming TTS:** The system must generate and stream synthesized speech for translated text.	MUST
FR-010	**Artifact Storage:** The system must store artifacts (audio, transcripts) in object storage (MinIO) for auditability.	MUST
FR-011	**Retention Policy:** The system must support automatic deletion of artifacts after a configurable retention period.	MUST
FR-012	**Event-Driven Orchestration:** Inter-service communication must be asynchronous via events. The workflow must be coordinated by a Pipeline Orchestrator to ensure ordering and state tracking.	MUST
FR-013	**Schema Governance:** Events must be governed by schemas (e.g., Avro) with strict compatibility checks.	MUST

ID	Description	Priority
FR-014	**Correlation:** A correlation ID must be propagated through all stages to trace user requests.	MUST
FR-015	**Retry & Degradation:** Services must retry on temporary failures and degrade gracefully on permanent ones.	MUST
FR-016	**Durability:** The system must resume processing after a restart without losing queued events.	MUST
FR-017	**Deduplication:** The client/UI should handle potential duplicate events (at-least-once delivery).	SHOULD
FR-018	**Docker Compose:** The system must be deployable locally via Docker Compose for development.	MUST
FR-019	**Docker Swarm:** The system must be deployable to a Docker Swarm cluster.	MUST
FR-020	**Horizontal Scaling:** Each microservice must be capable of independent horizontal scaling.	MUST
FR-021	**Ray Compute:** Compute-heavy stages (ASR, TTS) should utilize Ray Actors for warm-loaded execution.	SHOULD
FR-022	**Infrastructure as Code:** Infrastructure should be provisioned via Pulumi (Post-MVP).	SHOULD
FR-023	**Architectural Guardrails:** The system should implement runtime guardrails (e.g., filtering empty transcriptions, hallucination mitigation) to prevent invalid AI outputs from reaching the user.	SHOULD
FR-024	**VAD as Gatekeeper:** The system must utilize VAD to segment human speech from silence before sending data to compute-heavy ASR components.	MUST

A.2 Non-Functional Requirements

ID	Description	Priority
NFR-001	**Time-to-First-Output:** The first partial result must be delivered within 3 seconds.	MUST
NFR-002	**End-to-End Latency:** Median processing time per segment must be < 5 seconds.	MUST
NFR-003	**Scalability Test:** The system should support up to 20 concurrent sessions in testing.	SHOULD
NFR-004	**Observability:** The system must provide metrics, tracing, and structured logging.	MUST
NFR-005	**Privacy:** Logs must be anonymized and free of PII (raw audio/transcripts).	MUST

ID	**Description**	**Priority**
NFR-006	**Encryption:** Traffic should be encrypted (TLS) in transit (Post-MVP Requirement).	SHOULD
NFR-007	**Artifact Security:** Internal event references to artifacts must not contain bearer secrets.	MUST
NFR-008	**Resource Efficiency:** Services should run on standard server hardware with GPU support.	SHOULD
NFR-009	**Reproducibility:** Performance measurements must be reproducible via documented scripts.	MUST
NFR-010	**Quality Evaluation:** Translation quality is evaluated via both automated metrics and human rating.	SHOULD

B Sequence Diagrams

This appendix provides detailed sequence diagrams illustrating the asynchronous interaction between microservices.

C Data Contracts (Avro Schemas)

This appendix documents the strict data contracts enforced across the microservice pipeline using Apache Avro. These schemas serve as the "Shared Contract Artifact" described in Section 3.4, ensuring type safety and backward compatibility.

All events inherit a common envelope structure defined in `BaseEvent.avsc` to guarantee consistent tracing via `correlation_id`.

C.1 Base Envelope

The base structure that all specific events must adhere to. This ensures that infrastructure-level concerns (tracing, timing, provenance) are decoupled from business payloads.

```
{
  "type": "record",
  "name": "BaseEvent",
  "namespace": "ai.speech.shared.events",
  "doc": "Common envelope for all MVP events.",
  "fields": [
    {"name": "event_id", "type": "string", "doc": "UUID for this event"},
    {"name": "event_type", "type": "string", "doc": "Logical event type name"},
    {"name": "correlation_id", "type": "string", "doc": "UUID shared across pipeline"},
    {"name": "timestamp", "type": {"type": "long", "logicalType": "timestamp-millis"}, "doc": "Unix epoch ms"},
    {"name": "source_service", "type": "string", "doc": "Originating service name"}
  ]
}
```

Listing C.1: BaseEvent.avsc

C.2 Audio Ingestion

The `AudioInputEvent` is emitted by the Ingress Gateway. Note the `audio_bytes` field for small payloads and `audio_uri` for the "Claim Check" pattern if payloads exceed Kafka limits.

```
{
  "type": "record",
```

```
  "name": "AudioInputEvent",
  "namespace": "ai.speech.shared.events",
  "fields": [
    {"name": "event_id", "type": "string"},
    {"name": "event_type", "type": "string"},
    {"name": "correlation_id", "type": "string"},
    {"name": "timestamp", "type": {"type": "long", "logicalType": "timestamp-millis"}},
    {"name": "source_service", "type": "string"},
    {
      "name": "payload",
      "type": {
        "type": "record",
        "name": "AudioInputPayload",
        "fields": [
          {"name": "audio_bytes", "type": ["null", "bytes"], "default": null},
          {"name": "audio_uri", "type": ["null", "string"], "default": null},
          {"name": "audio_format", "type": "string", "doc": "e.g., wav"},
          {"name": "sample_rate_hz", "type": "int"},
          {"name": "language", "type": ["null", "string"], "default": null, "aliases": ["language_hint"]},
          {"name": "speaker_id", "type": ["null", "string"], "default": null}
        ]
      }
    }
  ]
}
```

Listing C.2: AudioInputEvent.avsc

C.3 Speech Segmentation (VAD)

The `SpeechSegmentEvent` represents a cleaned, verified chunk of speech ready for ASR. It includes timing information (`start_ms`, `end_ms`) relative to the session start.

```
{
  "type": "record",
  "name": "SpeechSegmentEvent",
  "namespace": "ai.speech.shared.events",
  "fields": [
    {"name": "event_id", "type": "string"},
    {"name": "correlation_id", "type": "string"},
    {
      "name": "payload",
      "type": {
        "type": "record",
        "name": "SpeechSegmentPayload",
```

```
13          "fields": [
14            {"name": "segment_id", "type": "string"},
15            {"name": "segment_index", "type": "int"},
16            {"name": "start_ms", "type": "long"},
17            {"name": "end_ms", "type": "long"},
18            {"name": "audio_bytes", "type": ["null", "bytes"], "
                default": null},
19            {"name": "segment_uri", "type": ["null", "string"], "
                default": null},
20            {"name": "sample_rate_hz", "type": "int"}
21          ]
22        }
23      }
24    ]
25  }
```

Listing C.3: SpeechSegmentEvent.avsc

C.4 Translation Output

The `TextTranslatedEvent` carries the core business value: the translated text. It includes source and target language codes to ensure downstream TTS picks the correct voice.

```
1  {
2    "type": "record",
3    "name": "TextTranslatedEvent",
4    "namespace": "ai.speech.shared.events",
5    "fields": [
6      {"name": "event_id", "type": "string"},
7      {"name": "correlation_id", "type": "string"},
8      {
9        "name": "payload",
10       "type": {
11         "type": "record",
12         "name": "TextTranslatedPayload",
13         "fields": [
14           {"name": "text", "type": ["null", "string"], "default":
               null},
15           {"name": "source_language", "type": "string"},
16           {"name": "target_language", "type": "string"},
17           {"name": "quality_score", "type": ["null", "float"], "
               default": null}
18         ]
19       }
20     }
21   ]
22 }
```

Listing C.4: TextTranslatedEvent.avsc

C.5 Synthesis Output (TTS)

The `AudioSynthesisEvent` is the final output. Similar to the input event, it supports both inline bytes and URIs via the Claim Check pattern to handle large generated audio files.

```
{
  "type": "record",
  "name": "AudioSynthesisEvent",
  "namespace": "ai.speech.shared.events",
  "fields": [
    {"name": "event_id", "type": "string"},
    {"name": "correlation_id", "type": "string"},
    {
      "name": "payload",
      "type": {
        "type": "record",
        "name": "AudioSynthesisPayload",
        "fields": [
          {"name": "audio_bytes", "type": ["null", "bytes"], "default": null},
          {"name": "audio_uri", "type": ["null", "string"], "default": null},
          {"name": "duration_ms", "type": "long"},
          {"name": "sample_rate_hz", "type": "int"},
          {"name": "audio_format", "type": "string", "default": "wav"}
        ]
      }
    }
  ]
}
```

Listing C.5: AudioSynthesisEvent.avsc

Bibliography

[1] Amazon Web Services. Amazon polly. https://aws.amazon.com/polly/, 2024. Accessed: 2026-01-26.

[2] Amazon Web Services. Amazon transcribe. https://aws.amazon.com/transcribe/, 2024. Accessed: 2026-01-26.

[3] Amazon Web Services. Amazon translate. https://aws.amazon.com/translate/, 2024. Accessed: 2026-01-26.

[4] Apache Kafka. Apache kafka documentation. https://kafka.apache.org/documentation/, 2023. Accessed: 2025-07-30.

[5] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.

[6] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley Professional, 4 edition, 2021.

[7] Len Bass, Qinghua Lu, Ingo Weber, and Liming Zhu. *Engineering AI Systems: Architecture and DevOps Essentials*. Addison-Wesley Professional, 2023.

[8] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. The MIT Press, Cambridge, Massachusetts, second edition, 2001.

[9] Google Cloud. Cloud translation. https://cloud.google.com/translate, 2024. Accessed: 2026-01-26.

[10] Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley Professional, 2004.

[11] Hirofumi Inaguma, Sravya Popuri, Gabriel Ilharco, Wei-Ning Hsu, Tal Remez, and Ann Lee. Unity: Two-pass direct speech-to-speech translation with discrete units. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 15315–15339, 2023.

[12] Yubin Kim, Ken Gu, Chanwoo Park, et al. Towards a science of scaling agent systems. *arXiv preprint arXiv:2512.08296*, 2025.

[13] Martin Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, 2017.

[14] Martin Kleppmann. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 2017.

[15] Kubernetes. Kubernetes documentation. https://kubernetes.io/docs/, 2023. Accessed: 2025-07-30.

[16] KUDO. 10 stats & breakthroughs in ai speech translation in 2024. https://kudo.ai/blog/the-10-most-important-statistics-breakthroughs-in-ai-speech-translatio 2024. Accessed: 2025-07-30, Blog post.

[17] B. P. Pradeep Kumar and Shreehari Menon. Real-time speech-to-speech translator: Analysis and implementation. In *2025 4th International Conference on Distributed Computing and Electrical Circuits and Electronics (ICDCECE)*. IEEE, 2025.

[18] Tim Lindholm and Frank Yellin. *The Java^{TM} Virtual Machine Specification*. Addison-Wesley, 2nd edition, 1999.

[19] Microsoft. Azure ai speech service. https://learn.microsoft.com/en-us/azure/ai-services/speech-service/, 2024. Accessed: 2026-02-13.

[20] Microsoft. Microsoft translator. https://www.microsoft.com/translator/, 2024. Accessed: 2026-01-26.

[21] Mordor Intelligence. Speech to speech translation market size & share analysis. https://www.mordorintelligence.com/industry-reports/speech-to-speech-translation, 2024. Accessed: 2025-07-30.

[22] Sam Newman. *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O'Reilly Media, 2019.

[23] Shaphaba Oinam, Bharat Sharma, Penumalli Dindi Venkat, and Pushpendra Ratnam Verma. Ai driven voice translator: Enhancing multilingual communication through real-time translation. In *2025 12th International Conference on Reliability, Infocom Technologies and Optimization (ICRITO)*. IEEE, 2025.

[24] Chen Qian, Zihao Xie, Yifei Wang, et al. Scaling large language model-based multi-agent collaboration. *arXiv preprint arXiv:2511.09525*, 2025.

[25] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. *arXiv preprint arXiv:2212.04356*, 2022.

[26] Chris Richardson. *Microservices Patterns: With Examples in Java*. Manning Publications, 2018.

[27] Mohammad Sarim, Saim Shakeel, Laeeba Javed, et al. Direct speech to speech translation: A review. *arXiv preprint arXiv:2406.07155*, 2024.

[28] UNESCO. Inclusion and education: All means all. https://www.unesco.org/, 2022. Accessed: 2026-01-26.

[29] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[30] Changhan Wang, Anne Wu, Juan Pino, Alexei Baevski, Michael Auli, and Alexis Conneau. Fairseq s2t: Fast speech-to-text modeling with fairseq. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 2020.

[31] Ping Xu, Aditi Sharma, and Jonas Müller. Scalability and maintainability challenges and solutions in machine learning: Systematic literature review. *arXiv preprint arXiv:2504.11079*, 2025.

[32] Y. Zhang et al. Direct speech-to-speech translation: Architecture and optimization, 2024. Preprint.